

# NeuroBridge: Shared Latent Dynamics, Spike Generation, and Representation Learning

Neuro\_Bridge

## 1 Purpose

NeuroBridge is a controlled framework for studying whether latent task structure can be recovered from neural population activity. The central idea is to define a known low-dimensional task dynamics, generate subject-specific spike-count observations from it, and then test whether representation-learning methods recover both the latent geometry and the temporal relationship between subjects.

$$\text{motor condition} \rightarrow Z \rightarrow U \rightarrow \lambda \rightarrow X \quad (1)$$

where:

- $Z$  is the low-dimensional latent trajectory.
- $U$  is the continuous neural drive.
- $\lambda$  is the positive firing-rate intensity.
- $X$  is the observed spike-count tensor.

## 2 Notation and Dimensions

Before specifying the model, we define the main mathematical objects.

| Symbol               | Meaning                           | General shape     |
|----------------------|-----------------------------------|-------------------|
| $n_{\text{trials}}$  | number of repeated trials         | scalar            |
| $L$                  | number of time bins in one trial  | scalar            |
| $\Delta t$           | duration of one bin               | scalar            |
| $k$                  | latent dimension                  | scalar            |
| $N$                  | number of neurons                 | scalar            |
| $c_i$                | condition/direction of trial $i$  | scalar            |
| $Z_i(t)$             | shared task latent state          | $k$ -vector       |
| $B^{(s)}$            | loading matrix for subject $s$    | $k \times N$      |
| $u_i^{(s)}(t)$       | neural drive                      | $N$ -vector       |
| $\lambda_i^{(s)}(t)$ | firing-rate intensity             | $N$ -vector       |
| $X_i^{(s)}(t)$       | spike-count vector                | $N$ -vector       |
| $W_i^{(s)}(t)$       | neural window given to an encoder | window $\times N$ |
| $E_i^{(s)}(t)$       | learned embedding                 | embedding vector  |

The full tensors are:

$$Z_{\text{task}} \in \mathbb{R}^{n_{\text{trials}} \times L \times k}, \quad (2)$$

$$X^{(s)} \in \mathbb{R}^{n_{\text{trials}} \times L \times N}, \quad (3)$$

$$B^{(s)} \in \mathbb{R}^{k \times N}, \quad (4)$$

$$E^{(s)} \in \mathbb{R}^{n_{\text{windows}} \times d_{\text{emb}}}. \quad (5)$$

Specific numerical values used in the poster run are reported later in the experimental-configuration section.

### 3 Latent Trajectories

The latent tensor has shape:

$$Z_{\text{task}} \in \mathbb{R}^{n_{\text{trials}} \times L \times k}. \quad (6)$$

For each trial  $i$ :

$$Z_i(t) = m_i(t) + \eta_i(t). \quad (7)$$

The deterministic part  $m_i(t)$  defines the expected task trajectory for a given condition. The stochastic part  $\eta_i(t)$  introduces smooth trial-to-trial variability around that expected trajectory.

#### 3.1 Deterministic Motor Component

For a circular task, each condition is associated with an angle:

$$\theta_i = \frac{2\pi(c_i - 1)}{n_{\text{conditions}}}, \quad (8)$$

where  $c_i$  is the condition label for trial  $i$ .

The temporal movement profile is:

$$s(t) = 10t^3 - 15t^4 + 6t^5, \quad (9)$$

with  $t \in [0, 1]$ .

The polynomial  $s(t)$  is a smooth phase variable for the trial. It maps the normalized trial time from 0 to 1 while keeping the endpoints flat:  $s(0) = 0$ ,  $s(1) = 1$ ,  $s'(0) = s'(1) = 0$ , and  $s''(0) = s''(1) = 0$ . In this simulator it is not meant to be a full biomechanical model of reaching; it is a controlled way to impose a smooth within-trial progression.

The deterministic latent trajectory is:

$$m_i(t) = \begin{bmatrix} s(t) \cos \theta_i \\ s(t) \sin \theta_i \\ s(t) \end{bmatrix}. \quad (10)$$

Therefore, the first two coordinates describe directional displacement on a circle, while the third coordinate stores the within-trial movement phase. This third coordinate is important: it prevents the latent trajectory from being only a static direction vector and gives the model an explicit temporal axis.

### 3.2 Stochastic Component

The noise term follows an AR(1)-like process:

$$\eta_i(t) = \phi\eta_i(t-1) + \epsilon_i(t), \quad (11)$$

where:

$$\epsilon_i(t) \sim \mathcal{N}(0, \sigma_\epsilon^2 I). \quad (12)$$

The innovation variance is  $\sigma_\epsilon^2$ . If the process is stationary and  $|\phi| < 1$ , the marginal variance of one latent-noise coordinate is:

$$\text{Var}[\eta(t)] = \frac{\sigma_\epsilon^2}{1 - \phi^2}. \quad (13)$$

Thus  $\phi$  controls temporal persistence, while  $\sigma_\epsilon$  controls the amount of trial-to-trial fluctuation injected at each time step.

## 4 Loading Matrix

The loading matrix specifies how the latent state is read out by a neural population. It is the bridge between task geometry and neuron-specific firing modulation:

$$B \in \mathbb{R}^{k \times N}. \quad (14)$$

Each column of  $B$  corresponds to one neuron:

$$\mathbf{b}_j = B_{:,j} \in \mathbb{R}^k. \quad (15)$$

### 4.1 Directional Tuning

The first two rows of  $B$  define preferred directions:

$$B_{1,j} = a \cos \varphi_j, \quad (16)$$

$$B_{2,j} = a \sin \varphi_j, \quad (17)$$

where  $a$  is the directional scale.

Neurons are grouped by preferred task condition. This means the population is structured: subsets of neurons preferentially respond to different directions. The directional scale controls how strongly the circular latent coordinates modulate the neural population. A larger value produces sharper directional tuning and stronger separation between movement directions.

The remaining latent dimensions, if present, receive smaller random loadings:

$$B_{3:k,j} \sim \alpha_{\text{extra}} \mathcal{N}(0, 1). \quad (18)$$

The parameter  $\alpha_{\text{extra}}$  controls weak sensitivity to non-directional latent coordinates. In the circular setting, it lets the movement-phase dimension influence firing rates without dominating the directional geometry.

## 4.2 Interpretation

The matrix  $B$  is not learned from data. It is imposed as part of the generative model.

Its purpose is to define how neurons read out the latent motor geometry. If a latent direction is aligned with the preferred direction of a neuron, that neuron receives higher drive.

## 5 Neural Drive

The continuous neural drive is:

$$U^{(s)} = Z^{(s)}B^{(s)} + \mathbf{c}^{(s)}. \quad (19)$$

With dimensions:

$$Z^{(s)} \in \mathbb{R}^{n_{\text{trials}} \times L \times k}, \quad (20)$$

$$B^{(s)} \in \mathbb{R}^{k \times N}, \quad (21)$$

$$\mathbf{c}^{(s)} \in \mathbb{R}^N, \quad (22)$$

$$U^{(s)} \in \mathbb{R}^{n_{\text{trials}} \times L \times N}. \quad (23)$$

For trial  $i$ , time  $t$ , and neuron  $j$ :

$$u_{i,t,j} = Z_i(t) \cdot \mathbf{b}_j + c_j. \quad (24)$$

### 5.1 Baseline

The baseline vector is:

$$\mathbf{c}^{(s)} = (c_1^{(s)}, \dots, c_N^{(s)}). \quad (25)$$

The baseline sets the firing-rate offset before the nonlinearity. In the poster simulation, all neurons use the same baseline drive. This keeps the first experiment simple, but neuron-specific baselines are a natural extension when matching real recordings.

## 6 Firing Rate

The neural drive  $U$  can be negative, so it cannot be used directly as a firing rate. A nonlinearity maps it into a positive rate:

$$\lambda_{i,t,j} = f(u_{i,t,j}). \quad (26)$$

The poster simulation uses the softplus nonlinearity:

$$f(x) = \log(1 + e^x). \quad (27)$$

The resulting tensor has shape:

$$\lambda^{(s)} \in \mathbb{R}^{n_{\text{trials}} \times L \times N}. \quad (28)$$

The rate  $\lambda_{i,t,j}$  is measured in expected spikes per unit time for neuron  $j$  at time  $t$  in trial  $i$ .

## 7 Spike-Count Generation

The base observation model samples spike counts as:

$$X_{i,t,j} \sim \text{Poisson}(\lambda_{i,t,j}\Delta t). \quad (29)$$

Here  $\lambda_{i,t,j}\Delta t$  is the expected spike count in one time bin. Thus:

$$X^{(s)} \in \mathbb{R}^{n_{\text{trials}} \times L \times N}. \quad (30)$$

The pure Poisson model assumes that, conditional on the rate, spike counts fluctuate randomly around their expected value. The temporal structure in  $X$  comes from the temporal structure in  $\lambda$ , which itself comes from  $Z$ .

The simulator also includes optional phenomenological extensions that make the observation model less idealized.

### 7.1 Overdispersion

Real spike counts often show variance larger than the mean. The pure Poisson model cannot represent this because it implies:

$$\text{Var}(X) = \mathbb{E}[X]. \quad (31)$$

To introduce overdispersion, the simulator can multiply the rate by a random Gamma gain:

$$\lambda_{i,t,j}^* = \lambda_{i,t,j} g_{i,t,j}, \quad (32)$$

where:

$$\mathbb{E}[g_{i,t,j}] = 1. \quad (33)$$

Spike counts are then sampled as:

$$X_{i,t,j} \sim \text{Poisson}(\lambda_{i,t,j}^* \Delta t). \quad (34)$$

Setting the overdispersion parameter to zero recovers the pure Poisson observation model.

### 7.2 Refractory Period

The simulator can approximate a refractory period at the bin level. After a nonzero spike count, the following bins for the same neuron and trial can be forced to zero for a sampled refractory duration.

The refractory duration is sampled as:

$$r_{\text{refractory}} \sim \max(0, \text{round}[\mathcal{N}(\mu_r, \sigma_r^2)]). \quad (35)$$

This is not a biophysical membrane model. It is a statistical correction that reduces very short inter-spike intervals.

### 7.3 Bursts and Local Spike Clusters

The simulator can also add short local spike clusters. A nonzero spike bin can start a burst with probability:

$$p_{\text{burst}}. \quad (36)$$

If a burst starts, extra spikes are sampled and added within a short temporal window.

This models local spike clustering phenomenologically, without attempting to simulate the cellular mechanisms that produce bursting.

## 8 Windowing

The spike tensor is reshaped:

$$X_{\text{reshaped}} \in \mathbb{R}^{(n_{\text{trials}}L) \times N}. \quad (37)$$

Temporal windows have length  $W = 10$  bins and are centered on each original time bin.

With centered padding, one window is produced for every original time bin. The missing samples at the beginning and end of each trial are filled with zeros. Thus the windowed dataset preserves the full time-series length:

$$n_{\text{windows}} = n_{\text{trials}}L. \quad (38)$$

This lets the encoder produce one embedding for each original time bin while still receiving a local temporal context. For lag analysis, each window keeps its trial identity, trial-local time index, and task condition. The condition assigned to a window is the condition of the trial from which the window was drawn.

Absolute trial-local time is used for lag-aware cross-population alignment. Relative time in  $[0, 1]$  is appropriate for behavioral similarity, but bin-level lag recovery requires integer trial-local time indices.

## 9 Distance and Similarity Construction

The circular reaching task uses two distances between windows:

- temporal distance, based on time
- circular condition distance, based on motor condition.

They are combined into a single task-informed distance. This means that two windows are considered close if they are close in trial phase, close in motor direction, or both.

More generally, the important distinction is not whether a variable is a “single label” or a “multi-label” target. The important distinction is the geometry of each behavioral variable. A behavioral variable can be:

- categorical, where equality or inequality is the relevant relation;
- circular, such as reaching direction on a circle;
- continuous one-dimensional, such as speed or position on a linear track;

- continuous multi-dimensional, such as a two-dimensional position  $(x, y)$ .

For a task containing both direction and two-dimensional position, the same framework can be written as:

$$D_{\text{total}} = w_t D_{\text{time}} + w_d D_{\text{circular}}(\text{direction}) + w_p D_{\text{euclidean}}(\text{position}_x, \text{position}_y). \quad (39)$$

The total distance is converted into a similarity matrix:

$$S = g(D_{\text{total}}). \quad (40)$$

This similarity matrix can be used as supervision or structure for downstream representation learning methods.

## 10 Representation Learning Objective

After windowing, each neural window is passed through an encoder:

$$z_i = f_{\theta}(X_i), \quad (41)$$

where  $X_i$  is a neural activity window and  $z_i \in \mathbb{R}^3$  is the learned representation. The poster comparison focuses on three models:

- PCA: a linear unsupervised baseline that projects neural activity onto directions of maximal variance;
- temporal CNN1D: a local temporal encoder that uses convolutional kernels to detect short-range spike-pattern structure within each window;
- Transformer: an attention-based temporal encoder that can relate different bins within the window.

The input convention for the neural encoders is a batch of windows with shape:

$$\text{batch} \times \text{features} \times \text{sequence length} = \text{batch} \times N \times \text{window}. \quad (42)$$

Thus the CNN is one-dimensional along time; neurons are treated as input features/channels.

### 10.1 Why These Three Encoders

The three encoders play different scientific roles. They should not be read as three arbitrary machine-learning models, but as three increasingly flexible ways of asking the same question: how much of the shared latent task structure can be recovered from population spike counts?

#### 10.1.1 PCA: Linear Variance-Based Baseline

Principal Component Analysis is the simplest reference model. Given a matrix of neural observations  $X \in \mathbb{R}^{m \times N}$ , where rows are observations and columns are neurons, PCA finds orthogonal directions  $v_1, \dots, v_d$  that maximize projected variance:

$$v_1 = \arg \max_{\|v\|=1} \text{Var}(Xv). \quad (43)$$

The low-dimensional representation is:

$$z_i = x_i V_d, \quad (44)$$

where  $V_d$  contains the first  $d$  principal axes. In the poster experiment  $d = 3$  when PCA is compared with the neural encoders.

PCA is useful because it is transparent. If PCA works well, then much of the latent structure is already expressed as large-scale linear covariance in the population activity. If PCA fails while nonlinear encoders work, then the latent structure may be present but not linearly aligned with the dominant variance directions.

Its limitation is equally important. PCA does not know about time, trial structure, task condition, or lag. It only sees variance. A high-variance direction may reflect nuisance variability rather than task geometry. Therefore, PCA is a baseline, not the main model.

### 10.1.2 Temporal CNN1D: Local Pattern Encoder

The temporal CNN1D receives a neural window  $X_i \in \mathbb{R}^{N \times W}$ , where  $N$  is the number of neurons and  $W$  is the window length in time bins. A one-dimensional convolution applies temporal filters across the window:

$$h_{r,t} = \sigma \left( b_r + \sum_{n=1}^N \sum_{\Delta=0}^{K-1} a_{r,n,\Delta} X_{n,t+\Delta} \right), \quad (45)$$

where  $r$  indexes filters,  $K$  is the convolutional kernel size, and  $\sigma$  is a nonlinearity. The key point is that the convolution runs along time. The neurons are input features/channels.

This model is well matched to the spike-window setting. It can detect local temporal patterns such as short increases in firing, local rate changes, and coordinated patterns across neurons inside the window. It also has an inductive bias: nearby time bins matter more directly than distant bins. This is useful when the latent state evolves smoothly and when the window is meant to capture local neural context around one time point.

In the current benchmark, the CNN1D is expected to be a strong compromise: more expressive than PCA, but less flexible than a Transformer. If it recovers a cleaner manifold and a plausible lag, that suggests the relevant information is local in time and can be captured by short temporal filters.

The main limitation is window scale. If the window is too short, the CNN sees too little temporal context. If the window is too long, the local filters may mix irrelevant parts of the trial. Thus window length is not just a technical hyperparameter; it is a modeling assumption about the temporal scale of neural information.

### 10.1.3 Transformer: Attention-Based Window Encoder

The Transformer treats the bins inside a window as a small sequence. Each time bin is mapped to a token representation, and self-attention computes how much each bin should attend to the others:

$$\text{Attention}(Q, K, V) = \text{softmax} \left( \frac{QK^\top}{\sqrt{d_k}} \right) V. \quad (46)$$

Unlike a convolution, attention is not restricted to nearby positions. Within the window, any bin can directly interact with any other bin. This makes the Transformer more flexible: it can learn



nonlocal relationships inside the window and can, in principle, detect temporal patterns that are not captured by fixed local kernels.

This flexibility is both a strength and a risk. A Transformer can align two subjects well in terms of lag while producing an embedding geometry that is compressed or visually less faithful to the original latent trajectory. In that case, the model may be learning a representation that is useful for cross-population consistency but not necessarily an interpretable recovery of the ground-truth latent space.

For this reason, the Transformer should be evaluated with multiple diagnostics: latent recovery, condition organization, Procrustes-aligned cross-population similarity, and lag recovery. A good lag score alone is not sufficient.

## 10.2 How To Read The Model Comparison

The three models answer different diagnostic questions:

- PCA asks whether the latent structure is visible through linear population covariance.
- CNN1D asks whether local temporal spike patterns are sufficient to recover task geometry and lag.
- Transformer asks whether a more flexible attention model can exploit within-window temporal relationships beyond local filters.

The best model is therefore not automatically the one with the best single metric. A useful model should recover the known latent geometry, preserve condition structure, and localize the imposed delay between the two populations. These criteria can disagree, and that disagreement is itself scientifically informative.

## 10.3 Soft Structured Contrastive Cross-Entropy

The main learning objective is a soft structured contrastive cross-entropy. Its role is not to reconstruct  $Z$  directly. Instead, task condition and trial-time variables define a target distribution over which neural windows should be close or far in the learned embedding.

For a batch of embeddings, the model computes cosine logits:

$$\ell_{ij} = \frac{\cos(z_i, z_j)}{\tau_{\text{temp}}}, \quad (47)$$

where  $\tau_{\text{temp}}$  is the contrastive temperature.

In a hard contrastive objective, a pair is usually either positive or negative. Here the target relationship is soft. A pair can be strongly similar, weakly similar, or dissimilar depending on task/time proximity. This is the same softmax-denominator logic used in InfoNCE, but with a soft target distribution instead of a single positive sample or a hard positive set.

The target similarity matrix  $S$  is obtained from a metadata distance:

$$S_{ij} = \exp\left(-\frac{D_{\text{total},ij}}{\tau_S}\right). \quad (48)$$

The total distance is a weighted combination of time and task-condition distances:

$$D_{\text{total},ij} = w_t D_{ij}^{\text{time}} + w_c D_{ij}^{\text{condition}}, \quad w_t + w_c = 1. \quad (49)$$

The time distance is:

$$D_{ij}^{\text{time}} = |t_i - t_j|. \quad (50)$$

The condition distance can be defined in different ways. For the current circular motor task, the run uses circular condition distance:

$$D_{ij}^{\text{condition}} = \min(|c_i - c_j|, n_{\text{conditions}} - |c_i - c_j|). \quad (51)$$

This choice makes neighboring directions more similar than opposite directions. If the intended supervision should only express same/different condition information, the appropriate alternative is categorical distance:

$$D_{ij}^{\text{condition}} = \begin{cases} 0, & c_i = c_j, \\ 1, & c_i \neq c_j. \end{cases} \quad (52)$$

This distinction is scientifically important. Circular distance injects a weak task geometry over conditions. Categorical distance only imposes class consistency and does not impose circular ordering.

For each anchor  $i$ , the row  $S_{i,:}$  is normalized into a target probability distribution over other batch elements. This target distribution is denoted  $q_{ij}$ :

$$q_{ij} = \frac{S_{ij}}{\sum_{k \neq i} S_{ik}}, \quad j \neq i. \quad (53)$$

The embedding logits define the model probability  $p_{ij}$ , i.e. the similarity distribution induced by the current learned embeddings:

$$p_{ij} = \frac{\exp(\ell_{ij})}{\sum_{k \neq i} \exp(\ell_{ik})}. \quad (54)$$

The loss is the cross-entropy between the soft target distribution  $q_{ij}$  and the embedding-induced distribution  $p_{ij}$ . For a minibatch of size  $B$ :

$$\mathcal{L}_{\text{soft-CE}} = -\frac{1}{B} \sum_{i=1}^B \sum_{\substack{j=1 \\ j \neq i}}^B q_{ij} \log p_{ij}. \quad (55)$$

Here  $B$  is the number of neural windows in the minibatch. The matrix  $Q = (q_{ij})$  is the target similarity distribution induced by task/time distances, while  $P = (p_{ij})$  is the similarity distribution induced by the current embeddings.

This objective is useful because it can use graded information: two windows can be partially similar if they are close in time, close in task condition, or both. It sits between a purely temporal InfoNCE objective and a hard supervised contrastive objective. Its main risk is that the learned geometry may reflect the chosen metadata weights too strongly. If the time term dominates, the embedding may mostly encode trial phase. If the condition term dominates, the embedding may separate conditions without preserving continuous trajectory geometry.

Therefore, this loss is task-structured contrastive learning. It is not a claim of purely unsupervised discovery: the simulator is used to test whether the representation learned under these constraints also recovers the known latent geometry and the known cross-population lag.

## 10.4 Supervised InfoNCE Baseline

The framework also supports a more standard supervised InfoNCE objective. In this case, positives are all samples in the batch with the same condition label:

$$\mathcal{P}(i) = \{j \neq i : y_j = y_i\}. \quad (56)$$

The loss is:

$$\mathcal{L}_{\text{supNCE}} = -\frac{1}{B} \sum_i \frac{1}{|\mathcal{P}(i)|} \sum_{p \in \mathcal{P}(i)} \log \frac{\exp(\cos(z_i, z_p)/\tau)}{\sum_{a \neq i} \exp(\cos(z_i, z_a)/\tau)}. \quad (57)$$

This objective is easier to communicate and closer to standard supervised contrastive learning. However, it treats all same-label samples as equally positive and ignores the circular distance between neighboring directions and the temporal structure of the trial.

## 10.5 Unsupervised Time-Offset InfoNCE

The framework also supports an unsupervised temporal baseline. In this objective, behavioral labels are not used. A positive pair is defined by temporal offset within the same trial:

$$\mathcal{P}_{\Delta}(i) = \{j : \text{trial}(j) = \text{trial}(i), |t_j - t_i| = \Delta\}. \quad (58)$$

The same InfoNCE denominator is then used over the batch. This is important as a baseline because it asks whether the network can recover a useful state space without explicit task labels.

# 11 What the Benchmark Tests

The benchmark separates three objects that are usually entangled in real data: the task-level latent geometry, the population-specific neural readout, and the stochastic spike-count observation. This separation is useful because the true latent trajectory is known. We can therefore ask whether a learned embedding recovers the latent geometry, whether it preserves condition structure, and whether it reveals the imposed temporal offset between the two populations.

The scientific question is not whether an embedding looks visually clean. The question is whether a model recovers information that was present in the controlled generative process:

- latent geometry: does the embedding preserve the low-dimensional task trajectory?
- condition structure: do trajectories remain organized by movement direction?
- temporal structure: does the embedding preserve within-trial phase?
- cross-subject structure: can two subject-specific neural populations be aligned back to a common task space?
- lag structure: can the known temporal delay between populations be recovered from the learned manifolds?

This distinction matters because a model can recover one part of the structure and fail on another. For example, it may localize the correct inter-subject lag while still producing a compressed or distorted latent geometry.

## 12 Multi-Subject Shared Latent Setting

The simulator can generate multiple subject-specific neural observations from a shared latent/task process. The screen stimulus, trial labels, and task-level latent trajectory are common to both subjects:

$$Z_{\text{task}}(t) \rightarrow X^{(1)}(t), X^{(2)}(t), \dots \quad (59)$$

Each subject has a different loading matrix, baseline, and spike observation noise:

$$U^{(s)} = Z^{(s)}B^{(s)} + \mathbf{c}^{(s)}. \quad (60)$$

The task labels and motor state are shared, but the neural response can be subject-specific. It is useful to distinguish two latent objects:

- $Z_{\text{task}}$ : the shared task/stimulus latent trajectory used as the ground-truth target for evaluation;
- $Z_{\text{neural-driver}}^{(s)}$ : the subject-specific latent driver, possibly lagged, used to generate neural activity.

This creates a controlled test of whether representation learning can recover a common latent/task geometry from different neural observations.

### 12.1 Temporal Lag

In interactive motor settings, two subjects may share the same task structure but show a small temporal offset in neural activity. For example, one subject may act while another observes or responds with delay.

The simulator represents this using a subject-specific neural response lag:

$$Z_{\text{neural-driver}}^{(s)}(t) = Z_{\text{task}}(t - \ell_s), \quad (61)$$

where  $\ell_s$  is measured in time bins.

The labels remain unchanged because both subjects see the same screen and task events. Only the neural driver used to generate spikes is shifted. This makes it possible to test whether learned embeddings remain alignable when subject-specific neural observations are slightly shifted in time.

## 13 Shared-Space Alignment

If two encoders recover the same latent structure, their coordinates need not match exactly. Latent representations are often identifiable only up to geometric transformations such as translation, rotation, reflection, scaling, axis permutation, mild nonlinear warping, and temporal lag.

Therefore, cross-subject evaluation should not require raw coordinate equality. Instead, it should test whether the recovered geometries are equivalent after reasonable alignment.

### 13.1 Recommended Evaluation

A conservative evaluation stack is:

- center and scale embeddings;
- align embeddings using orthogonal Procrustes;
- measure residual alignment error;
- compare representational distance matrices;
- compute latent recovery against the known  $Z$ ;
- search over small temporal lags when the task implies delayed interaction.

More flexible methods, such as affine alignment, CCA, CKA, dynamic time warping, manifold alignment, optimal transport, or Gromov–Wasserstein alignment, can be useful later. However, overly flexible alignment can hide real failures of the representation model. The first benchmark should therefore start with Procrustes alignment and representational similarity analysis.

### 13.2 Lag-Aware Procrustes Alignment

For cross-subject comparison, the current baseline estimates the temporal offset by scanning a small set of candidate lags:

$$\ell \in \{-15, \dots, 0, \dots, 15\}. \quad (62)$$

For each candidate lag, the comparison pairs:

$$z^{(1)}(\text{trial}, t) \quad \text{with} \quad z^{(2)}(\text{trial}, t + \ell). \quad (63)$$

The second subject embedding is then aligned to the first subject embedding using orthogonal Procrustes, and the lag with the best Procrustes  $R^2$  is selected:

$$\ell^* = \arg \max_{\ell} R_{\text{Procrustes}}^2(\ell). \quad (64)$$

In the current synthetic configuration, the delayed population is generated with a ten-bin neural-response lag:

$$\ell_{\text{delayed}} = 10 \quad \text{bins} = 100 \quad \text{ms}. \quad (65)$$

The CNN gives the most sensible trade-off between latent geometry recovery and lag localization. The Transformer can peak at the correct lag while producing a compressed and less faithful geometry. This dissociation is important: cross-subject temporal consistency and ground-truth latent recovery are not the same validation target.

The estimated lag is selected over candidate lags, not assumed.

### 13.3 Poster Pipeline: From Spikes to Temporal Asymmetry

The complete poster pipeline can be written as:

$$Z_{\text{task}} \rightarrow \left( Z_{\text{driver}}^{(\text{earlier})}, Z_{\text{driver}}^{(\text{delayed})} \right) \quad (66)$$

$$\rightarrow \left( U^{(\text{earlier})}, U^{(\text{delayed})} \right) \rightarrow \left( \lambda^{(\text{earlier})}, \lambda^{(\text{delayed})} \right) \quad (67)$$

$$\rightarrow \left( X^{(\text{earlier})}, X^{(\text{delayed})} \right) \rightarrow \text{windows} \rightarrow \left( E^{(\text{earlier})}, E^{(\text{delayed})} \right) \quad (68)$$

$$\rightarrow \text{lag diagnostics}. \quad (69)$$

The shared latent is:

$$Z_{\text{task}} \in \mathbb{R}^{200 \times 100 \times 3}. \quad (70)$$

The subject-specific neural drivers are:

$$Z_{\text{driver}}^{(\text{earlier})}(t) = Z_{\text{task}}(t), \quad (71)$$

$$Z_{\text{driver}}^{(\text{delayed})}(t) = Z_{\text{task}}(t - 10). \quad (72)$$

Thus the delayed population is shifted by 10 bins, corresponding to 100 ms.

The spike tensors are:

$$X^{(\text{earlier})} \in \mathbb{R}^{200 \times 100 \times 80}, \quad (73)$$

$$X^{(\text{delayed})} \in \mathbb{R}^{200 \times 100 \times 80}. \quad (74)$$

The encoders operate on center-padded neural windows:

$$W_i^{(s)} \in \mathbb{R}^{10 \times 80}. \quad (75)$$

Each window is mapped to a three-dimensional embedding:

$$E_i^{(s)} = f_{\theta}(W_i^{(s)}), \quad E_i^{(s)} \in \mathbb{R}^3. \quad (76)$$

Because the stride is one and padding preserves trial length, each subject has:

$$200 \times 100 = 20000 \quad (77)$$

window embeddings.

### 13.4 Manifold-Level Temporal Diagnostic

Beyond the Procrustes lag scan, the recovered manifolds can be inspected with a lag-lag diagnostic. The goal is to ask whether temporal structure in the earlier population appears shifted relative to temporal structure in the delayed population.

For each population, the recovered embedding sequence is divided into temporal blocks. For block  $b$ , let:

$$M_b^{(\text{earlier})}, M_b^{(\text{delayed})} \in \mathbb{R}^{n_{\text{obs}} \times d_{\text{emb}}}, \quad (78)$$

where rows are trial/block observations and columns are embedding dimensions. A simple diagnostic compares block summaries across populations:

$$L_{b,b'} = \text{sim} \left( M_b^{(\text{earlier})}, M_{b'}^{(\text{delayed})} \right). \quad (79)$$

In the current exploratory version, the similarity can be a correlation between block-level summaries. This should be interpreted cautiously: it is a visual diagnostic of temporal asymmetry, not a causal coupling measure.

## 14 Poster Storyline

This section explains how to read and present the poster. The goal is not only to describe the figures, but to make clear what scientific claim each block is allowed to support.

### 14.1 Question

The poster starts from one question: if two neural populations are driven by the same task-level latent process, but through different neural readouts and with a small temporal delay, can representation learning recover a shared geometry and reveal the delay?

The important point is that the shared latent space is known because the data are synthetic. This gives a controlled benchmark. We are not simply asking whether an embedding looks organized. We are asking whether it recovers a known ground-truth structure and whether temporal asymmetry can be detected after encoding.

### 14.2 Workflow

The workflow summarizes the full causal chain used in the experiment:

$$\text{shared task latent} \rightarrow \text{subject-specific readouts} \rightarrow \text{spike populations} \rightarrow \text{windows} \quad (80)$$

$$\rightarrow \text{encoders} \rightarrow \text{recovered manifolds and lag diagnostics}. \quad (81)$$

This order matters. The two populations do not start from identical spike matrices. They share the same task-level latent process, but each population has its own neural loading matrix and stochastic spike generation. Therefore, any recovered similarity between the populations must survive the subject-specific neural readout and the spike-count observation process.

### 14.3 Generative Block

The generative block explains how the synthetic neural data are created. The shared latent trajectory is denoted by  $Z$ . For each population,  $Z$  is mapped to a neural drive:

$$U = ZB + \mathbf{c}, \quad (82)$$

where  $B$  is a population-specific loading matrix and  $\mathbf{c}$  is an offset. The drive is transformed into a positive firing intensity through a softplus nonlinearity:

$$\lambda = \text{softplus}(U). \quad (83)$$

Spike counts are then sampled from a Poisson observation model:

$$X \sim \text{Poisson}(\lambda\Delta t), \quad (84)$$

with optional phenomenological corrections such as overdispersion, refractoriness, and short burst-like clusters. The delayed population is generated from a temporally shifted neural driver. In the poster run, the delay is ten bins, corresponding to 100 ms.

#### 14.4 Panels A and B

Panel A shows the ground-truth latent trajectories. These are the trajectories that the learning models should recover from spikes. The colors correspond to motor directions. Because  $Z$  is known, the figure is not an exploratory visualization: it is the target geometry of the benchmark.

Panel B shows observed spike rasters from the two generated populations. This panel is deliberately much less structured visually than Panel A. It reminds the reader that the encoder does not receive the latent trajectory directly. It receives noisy, binned spike counts generated through population-specific readouts.

#### 14.5 Learning Block

The learning block is the methodological center of the poster. Each spike matrix is converted into local temporal windows. In the poster configuration, each window contains ten time bins around a target bin and all recorded neurons. The encoder maps each window to a low-dimensional embedding:

$$e_i = f_\theta(W_i), \quad e_i \in \mathbb{R}^3. \quad (85)$$

Three encoder families are compared:

- PCA, which asks whether the latent geometry is already visible in linear population covariance;
- CNN1D, which asks whether local temporal spike patterns are sufficient to recover geometry and delay;
- Transformer, which asks whether attention over the window can exploit more flexible temporal relationships.

The contrastive target is not a hard same/different label. Instead, a metadata distance is built from trial time and task condition:

$$D_{ij} = w_t D_{ij}^{\text{time}} + w_c D_{ij}^{\text{condition}}. \quad (86)$$

This distance is converted into a soft target distribution:

$$q_{ij} \propto \exp(-D_{ij}/\tau_S). \quad (87)$$

The embedding itself induces a predicted similarity distribution:

$$p_{ij} \propto \exp(\cos(e_i, e_j)/\tau). \quad (88)$$

Training minimizes cross-entropy between the target distribution  $Q$  and the embedding-induced distribution  $P$ :



$$\mathcal{L}_{\text{soft-CE}} = -\frac{1}{B} \sum_{i=1}^B \sum_{\substack{j=1 \\ j \neq i}}^B q_{ij} \log p_{ij}. \quad (89)$$

This is a task-structured contrastive objective. It is related to InfoNCE because it uses a softmax distribution over other samples in the batch, but it uses a soft target distribution rather than a single positive sample.

## 14.6 Panels C and D

Panels C and D show recovered manifolds from the CNN encoder. Panel C is shown in three dimensions because the encoder output has dimension three and the goal is to inspect the learned geometry in its native output space. Panel D is shown as a two-dimensional projection for readability: it gives a cleaner view of condition organization for the delayed population.

The difference between a 3D view and a 2D projection should not be overinterpreted. A 2D projection is a visualization choice. The model output can still be three-dimensional even when the plotted figure displays only two axes or a projection. The key question is whether the recovered trajectories preserve condition structure and resemble the known latent geometry.

## 14.7 Temporal Recovery

Temporal recovery asks whether the delay imposed during generation can be estimated from the learned embeddings. The earlier and delayed populations are not compared point-by-point without adjustment. Instead, the delayed embedding is shifted by a set of candidate lags:

$$\ell \in \{-15, \dots, 15\}. \quad (90)$$

For each candidate lag, the delayed embedding is paired with the earlier embedding at the corresponding shifted time. The two embeddings are then aligned with orthogonal Procrustes.

## 14.8 Procrustes Alignment

Procrustes alignment answers a geometric question: are two recovered spaces the same up to a rigid transformation? Given two centered matrices  $A$  and  $B$ , Procrustes finds the orthogonal matrix  $R$  that minimizes:

$$\|A - BR\|_F^2 \quad \text{subject to} \quad R^\top R = I. \quad (91)$$

This allows rotation and reflection, but does not allow arbitrary warping. That is why it is useful as a conservative alignment diagnostic. If two embeddings align well after Procrustes, they share a similar geometry even if their axes are not identical.

In the lag scan, this alignment is repeated for every candidate lag. The selected lag is the lag with the best Procrustes alignment score:

$$\ell^* = \arg \max_{\ell} R_{\text{Procrustes}}^2(\ell). \quad (92)$$

## 14.9 Panel E: Lag Recovery Scan

Panel E shows the alignment score as a function of candidate lag. The vertical reference line marks the true imposed delay of ten bins. A model that peaks near this value has recovered the temporal offset between the two populations.

However, this panel should be interpreted together with the manifold plots. A model can recover the correct lag while still producing a distorted or compressed geometry. Therefore, lag recovery is necessary but not sufficient for claiming faithful latent recovery.

### 14.10 Lag-Lag Map

The lag-lag map is a complementary temporal diagnostic. Instead of selecting a single candidate delay, both recovered manifolds are divided into temporal blocks. For each earlier block  $b$  and delayed block  $b'$ , a similarity score is computed:

$$L_{b,b'} = \text{sim} \left( M_b^{(\text{earlier})}, M_{b'}^{(\text{delayed})} \right). \quad (93)$$

This produces a matrix of temporal correspondences. If the two populations were perfectly synchronous, the strongest structure would be expected near the main diagonal. If one population is delayed, the structure may shift away from the diagonal. In the current poster this is exploratory: it is meant to visualize temporal asymmetry, not to prove causal coupling.

### 14.11 Panel F: Exploratory Lag-Lag Map

Panel F shows this block-wise temporal comparison. The map should be read as a summary of how temporal profiles in one recovered manifold relate to temporal profiles in the other. Warm or cool regions indicate positive or negative correspondence between blocks, depending on the chosen similarity measure.

The main interpretive point is modest: the lag-lag map can reveal temporal structure that is not visible in a single static embedding plot. It is not yet a decoder, not a causal analysis, and not a replacement for cross-validated prediction.

### 14.12 Interpretation and Required Controls

The current interpretation is that the benchmark can separate three questions:

- whether the latent task geometry is recoverable from spikes;
- whether the two population embeddings can be aligned into a common space;
- whether the imposed delay can be estimated from the recovered manifolds.

Before making stronger claims, several controls are required:

- a lag-zero control, where both populations are generated without temporal delay;
- repeated simulations across multiple random seeds;
- time-only and condition-only loss ablations;
- shuffled-condition and shuffled-time controls;

- comparison between PCA, CNN1D, and Transformer under identical training and evaluation settings;
- cross-validated prediction from one population embedding to the other population, which would be stronger than a visual lag-lag map.

These controls are essential because a visually appealing manifold is not enough. The goal is to show that the recovered geometry and temporal asymmetry are stable consequences of the generative structure and learning objective, not artifacts of plotting, seed choice, or overly strong metadata supervision.

## 15 Operational Questions For The Poster Discussion

This section collects the questions that are most likely to arise during a poster discussion. The emphasis is practical: what object is manipulated, what the model receives, and what each plot is allowed to mean.

### 15.1 What Does It Mean To Lag One Population?

In the poster simulation, the delayed population is not delayed by 10 milliseconds. It is delayed by 10 time bins. Since the bin size is  $\Delta t = 10$  ms, this corresponds to:

$$10 \text{ bins} \times 10 \text{ ms/bin} = 100 \text{ ms.} \quad (94)$$

The lag is applied at the level of the latent neural driver, before spike generation. The shared task trajectory is  $Z_{\text{task}}(t)$ . The earlier population uses:

$$Z_{\text{driver}}^{(\text{earlier})}(t) = Z_{\text{task}}(t), \quad (95)$$

whereas the delayed population uses:

$$Z_{\text{driver}}^{(\text{delayed})}(t) = Z_{\text{task}}(t - 10). \quad (96)$$

Intuitively, the delayed population is driven by the same latent trajectory, but it is seeing an earlier state of that trajectory at the current time bin. If the earlier population is at latent state  $Z(t)$ , the delayed population is approximately at the state that the earlier population had ten bins before.

At the beginning of the trial, there is no time  $t - 10$ . The implementation does not wrap the trajectory around the trial. Instead, it pads the beginning using the edge value of the trial. In code, the positive-lag case is equivalent to:

$$Z_{\text{lagged}}[:, \ell, :] = Z[:, 1, :], \quad (97)$$

$$Z_{\text{lagged}}[:, \ell :, :] = Z[:, :, -\ell, :]. \quad (98)$$

Thus the lag is within trial, not across trials. Trial labels and task conditions remain the same for both populations, because both populations are assumed to share the same external task events. What changes is the neural response timing.

### 15.1.1 Matrix View Of The Lag

For one trial, ignore the trial index and write the latent trajectory as a matrix:

$$Z = \begin{bmatrix} z_1 \\ z_2 \\ \dots \\ z_L \end{bmatrix} \in \mathbb{R}^{L \times k}, \quad z_t \in \mathbb{R}^k. \quad (99)$$

With a positive lag of  $\ell$  bins, the delayed trajectory is:

$$Z^{(\text{delayed})} = \begin{bmatrix} z_1 \\ \dots \\ z_1 \\ z_1 \\ z_2 \\ \dots \\ z_{L-\ell} \end{bmatrix} \in \mathbb{R}^{L \times k}, \quad (100)$$

where the first  $\ell$  rows are filled by the first valid latent state because the current implementation uses edge padding. The important point is that the shape does not change. The earlier and delayed populations still have latent tensors with shape:

$$n_{\text{trials}} \times L \times k. \quad (101)$$

The delayed population is therefore not a different task and not a different label sequence. It is the same task-related latent trajectory sampled with a response delay before generating spikes.

## 15.2 What Is One Observational Unit?

The raw spike tensor for one population has shape:

$$X \in \mathbb{R}^{n_{\text{trials}} \times L \times N}, \quad (102)$$

where  $L$  is the number of time bins per trial and  $N$  is the number of neurons. In the poster configuration:

$$n_{\text{trials}} = 200, \quad L = 100, \quad N = 80. \quad (103)$$

The learning models do not receive an entire trial as one sample. They receive local windows. With window size  $W = 10$ , one observational unit is:

$$W_i \in \mathbb{R}^{10 \times 80}. \quad (104)$$

This means: ten consecutive time bins, all neurons. With centered padding and stride one, every original time bin has one associated window. Therefore one population yields:

$$200 \times 100 = 20000 \quad (105)$$

windows and therefore 20000 embeddings.

### 15.3 What Is A Batch?

A batch is a collection of windows sampled for one optimization step. If the batch size is  $B$ , the model receives:

$$\text{batch} \in \mathbb{R}^{B \times 10 \times 80}. \quad (106)$$

So, if  $B = 30$ , then the batch contains 30 windows, and each window contains 10 time bins by 80 neurons. In the current baseline suite the batch size is a hyperparameter; for example, one configuration uses  $B = 256$ . The important point is:

$$\text{batch size} \neq \text{window size}. \quad (107)$$

The window size says how much local temporal context each sample contains. The batch size says how many such samples are processed together and compared inside the contrastive loss.

### 15.4 How Does Each Model See The Same Window?

All models receive windows derived from the same spike data, but they interpret the window differently.

#### 15.4.1 PCA

PCA is not temporal in itself. To apply PCA to a window, the window is treated as a vector of features. A  $10 \times 80$  window can be flattened into:

$$\text{vec}(W_i) \in \mathbb{R}^{800}. \quad (108)$$

PCA then finds linear directions of maximal variance across all windows. It does not know that the first ten groups of features came from ordered time bins unless this temporal structure is already reflected in the covariance.

Therefore, PCA is useful as a baseline: it asks whether the latent structure is already visible in linear variance. But PCA does not learn temporal filters, attention, or task-aware nonlinear transformations.

#### 15.4.2 CNN1D

The CNN1D treats the window as a short temporal signal. Conceptually, the input is arranged as:

$$\text{batch} \times \text{neurons} \times \text{time} = B \times 80 \times 10. \quad (109)$$

The convolution slides filters along the time dimension. A filter can detect local temporal motifs such as a short rise in firing rate, a local pattern across neurons, or a brief coordinated event. This is why the window is most natural for the CNN: the model is explicitly designed to exploit local temporal structure.

The CNN has a strong inductive bias. It assumes that nearby bins contain useful information and that local temporal patterns can be reused across the window. This can be an advantage when neural dynamics are smooth or locally structured. It can be a limitation if the relevant relationship is nonlocal or depends on longer trial context.

### 15.4.3 Transformer

The Transformer also receives the same  $10 \times 80$  window, but interprets it as a sequence of ten tokens. Each token corresponds to one time bin, whose features are the neural population activity at that bin:

$$W_i = (x_{t-4}, x_{t-3}, \dots, x_{t+5}), \quad x_t \in \mathbb{R}^{80}. \quad (110)$$

Self-attention lets each time bin compare itself with every other bin inside the window:

$$\text{Attention}(Q, K, V) = \text{softmax} \left( \frac{QK^\top}{\sqrt{d_k}} \right) V. \quad (111)$$

This means the Transformer is less local than the CNN. It can learn that the beginning and end of the window are related, or that some bins should be weighted more than others. This flexibility can help with temporal alignment, but it can also produce embeddings that are useful for the lag score while being geometrically compressed or harder to interpret.

### 15.4.4 What A Token Means Here

In language models, a token is usually a word fragment. In this neural setting, a token is not a word. A token is one time bin of neural population activity. For a window with  $W = 10$  bins and  $N = 80$  neurons, the input window is:

$$W_i = \begin{bmatrix} x_{t-4} \\ x_{t-3} \\ \dots \\ x_{t+5} \end{bmatrix}, \quad x_t \in \mathbb{R}^{80}. \quad (112)$$

Thus the Transformer sees ten tokens:

$$\underbrace{x_{t-4}}_{\text{token 1}}, \underbrace{x_{t-3}}_{\text{token 2}}, \dots, \underbrace{x_{t+5}}_{\text{token 10}}. \quad (113)$$

Each token is an 80-dimensional vector containing the activity of the whole population at one time bin. A linear layer first maps each token from  $\mathbb{R}^{80}$  to the internal Transformer dimension  $d_{\text{model}}$ . Self-attention then lets the ten bins inside the same window compare with one another. After attention, the window is summarized into one output embedding:

$$f_\theta(W_i) = e_i, \quad e_i \in \mathbb{R}^3. \quad (114)$$

This is the crucial dimensional point: the Transformer does not output one embedding per token in the final representation used for the contrastive loss. It uses the ten tokens to compute one representation for the whole local window.

### 15.4.5 Do Windows Talk To Each Other Inside The Transformer?

In the current setup, no. The Transformer operates inside one window at a time. If the batch has shape:

$$B \times 10 \times 80, \quad (115)$$

then the Transformer processes each of the  $B$  windows independently and returns:

$$E_{\text{batch}} \in \mathbb{R}^{B \times 3}. \quad (116)$$

The communication between different windows happens in the loss, not inside the Transformer. This gives two separate levels:

- within-window learning: CNN filters or Transformer attention extract a representation from the ten-bin neural context;
- between-window learning: the contrastive loss compares the resulting embeddings across windows in the batch.

So the model first converts each local neural movie  $W_i \in \mathbb{R}^{10 \times 80}$  into one point  $e_i \in \mathbb{R}^3$ . Then the loss asks whether the distances among those points are compatible with the task/time metadata distances among the corresponding windows.

## 15.5 Why Use Windows For PCA And Transformer Too?

Using windows for all models keeps the comparison fair: each method receives the same local neural information around each target time bin. The difference is not the data each method sees, but how the method processes that data.

For PCA, the window is a high-dimensional vector. For CNN1D, the window is a local temporal signal. For the Transformer, the window is a short sequence. Thus the same window supports three different modeling assumptions:

- linear covariance structure for PCA;
- local temporal filters for CNN1D;
- flexible within-window temporal relations for the Transformer.

## 15.6 What Exactly Does Procrustes Do In The Lag Plot?

The Procrustes step is not used to train the model. It is used after training as an evaluation tool. Suppose we have two embeddings:

$$E^{(\text{earlier})} \quad \text{and} \quad E^{(\text{delayed})}. \quad (117)$$

Even if both embeddings recover the same latent geometry, their coordinate axes may be rotated or reflected relative to each other. Procrustes asks: can one embedding be rotated/reflected so that it overlaps the other?

For a candidate lag  $\ell$ , we first pair points by trial and shifted time:

$$E^{(\text{earlier})}(\text{trial}, t) \leftrightarrow E^{(\text{delayed})}(\text{trial}, t + \ell). \quad (118)$$

Then we find the best orthogonal transformation  $R_\ell$ :

$$R_\ell = \arg \min_{R^\top R = I} \left\| E^{(\text{earlier})} - E^{(\text{delayed})} R \right\|_F^2. \quad (119)$$

The score plotted in the lag recovery graph is high when the delayed embedding, after the candidate shift and Procrustes rotation, matches the earlier embedding well. The best lag is:

$$\ell^* = \arg \max_{\ell} R_{\text{Procrustes}}^2(\ell). \quad (120)$$

Intuitively, the graph says: “if I shift the delayed population by this many bins, how well can its recovered manifold be rotated onto the earlier population’s manifold?” A peak near the true lag suggests that the embedding contains the temporal offset. It does not by itself prove that the entire latent geometry was recovered faithfully.

### 15.6.1 What Is Lost When Testing A Candidate Lag?

Suppose the two recovered embeddings are:

$$E^{(\text{earlier})}, E^{(\text{delayed})} \in \mathbb{R}^{20000 \times 3}. \quad (121)$$

The number 20000 is the number of window-centered observations, for example 200 trials times 100 bins. The number 3 is the embedding dimension.

If we test a candidate lag of  $\ell = 15$  bins, we cannot compare all 20000 rows, because the shifted sequence has missing partners at the edges of each trial. For a positive lag, the comparison is conceptually:

$$E^{(\text{earlier})}(\text{trial}, t) \quad \text{with} \quad E^{(\text{delayed})}(\text{trial}, t + \ell). \quad (122)$$

Only valid within-trial pairs are kept. Therefore, some rows near the beginning or end of each trial are discarded for that candidate lag. What is discarded is time points, not embedding dimensions. The matrices passed to Procrustes are:

$$A_{\ell}, B_{\ell} \in \mathbb{R}^{m_{\ell} \times 3}, \quad (123)$$

where  $m_{\ell}$  is the number of valid matched rows after shifting. The three embedding coordinates are still present.

### 15.6.2 What Procrustes Maximizes

For each candidate lag  $\ell$ , Procrustes finds the best rotation/reflection of the delayed embedding onto the earlier embedding:

$$R_{\ell} = \arg \min_{R^{\top} R = I} \|A_{\ell} - B_{\ell} R\|_F^2. \quad (124)$$

The associated score can be expressed as an alignment  $R^2$ : how much of the variance in  $A_{\ell}$  is explained by the optimally aligned  $B_{\ell}$ . The lag scan repeats this for many candidate lags and selects:

$$\ell^* = \arg \max_{\ell} R^2(\ell). \quad (125)$$

Thus the lag score is not a nonlinear decoder. It is a lag-conditioned linear geometric alignment test. It asks whether, after matching the correct temporal rows and allowing a rigid change of coordinate system, the two recovered manifolds occupy the same geometry.



## 15.7 How To Explain The Soft Contrastive Loss

The loss builds a target notion of similarity between windows before looking at the embedding. Two windows are treated as similar if they are close in trial time, close in task condition, or both. This is encoded by a distance:

$$D_{ij} = w_t D_{ij}^{\text{time}} + w_c D_{ij}^{\text{condition}}. \quad (126)$$

This distance is transformed into a soft probability distribution:

$$q_{ij} = \frac{\exp(-D_{ij}/\tau_S)}{\sum_{k \neq i} \exp(-D_{ik}/\tau_S)}. \quad (127)$$

The embedding produces its own probability distribution:

$$p_{ij} = \frac{\exp(\cos(e_i, e_j)/\tau)}{\sum_{k \neq i} \exp(\cos(e_i, e_k)/\tau)}. \quad (128)$$

The loss minimizes:

$$\mathcal{L}_{\text{soft-CE}} = -\frac{1}{B} \sum_{i=1}^B \sum_{j \neq i} q_{ij} \log p_{ij}. \quad (129)$$

In words: if the metadata say that two windows should be close, the embedding is encouraged to assign them high similarity. If the metadata say they should be far, the embedding is encouraged to assign them low similarity. This is “soft” because similarity is graded, not binary. It is contrastive because each window is compared against other windows in the batch.

### 15.7.1 Anchors, Soft Positives, And Effective Negatives

In a standard hard contrastive setup, one often chooses an anchor, one positive sample, and many negative samples. Here the logic is softer.

For a batch of  $B$  windows, the encoder returns:

$$E_{\text{batch}} = \begin{bmatrix} e_1 \\ e_2 \\ \dots \\ e_B \end{bmatrix} \in \mathbb{R}^{B \times 3}. \quad (130)$$

For each row  $i$ , the embedding  $e_i$  is the anchor. Every other row  $j \neq i$  is a comparison candidate. The metadata distance  $D_{ij}$  decides how much candidate  $j$  should behave like a positive example for anchor  $i$ :

- small  $D_{ij}$ : window  $j$  is a strong soft positive for window  $i$ ;
- intermediate  $D_{ij}$ : window  $j$  is partially related to window  $i$ ;
- large  $D_{ij}$ : window  $j$  behaves like an effective negative.

Thus the loss does not say “same condition is positive, different condition is negative” in a hard binary way. It says that the whole similarity profile around anchor  $i$  should match the metadata-defined similarity profile around that same anchor. This is why the loss can combine time and condition without collapsing everything into a single hard label.

### 15.7.2 Where The Window Ends And The Embedding Begins

The model never compares raw windows directly in the loss. The comparison is between embeddings. The complete pipeline for one batch is:

$$W_i \in \mathbb{R}^{10 \times 80} \xrightarrow{f_\theta} e_i \in \mathbb{R}^3 \xrightarrow{\text{batch comparison}} \{p_{ij}\}_{j \neq i}. \quad (131)$$

So, yes: the encoder first studies structure inside each window. Then the loss studies relations between windows by comparing their 3D embeddings. The metadata distance is defined between windows because each embedding corresponds to one window.

## 15.8 How To Explain The Lag-Lag Map

The lag recovery scan searches for a single best delay. The lag-lag map asks a more exploratory question: how do temporal blocks in one recovered manifold relate to temporal blocks in the other?

Each recovered embedding sequence is divided into temporal blocks. For each pair of blocks, one from the earlier population and one from the delayed population, the analysis computes a similarity:

$$L_{b,b'} = \text{sim} \left( M_b^{(\text{earlier})}, M_{b'}^{(\text{delayed})} \right). \quad (132)$$

Rows and columns are temporal blocks or lags. A diagonal pattern suggests that similar temporal structure occurs at similar times. A shifted pattern suggests that one population’s temporal structure is displaced relative to the other.

The limitation is important: in the current poster, this is a descriptive diagnostic. It summarizes temporal correspondence between recovered manifolds, but it is not yet a decoding proof, not a causal measure, and not a statistical test of coupling.

## 15.9 Limits To State Clearly

The main limits to state in discussion are:

- the data are synthetic, so the current claim is about benchmark behavior, not direct biological discovery;
- the delay is imposed in the latent neural driver, not estimated from real simultaneous recordings;
- the soft contrastive loss injects task/time structure through the metadata distance, so the result is task-structured learning, not pure unsupervised discovery;
- a correct lag peak does not guarantee faithful recovery of the full latent geometry;
- the lag-lag map is exploratory and should be strengthened with cross-validated prediction or decoding;
- window size, metadata weights, temperature, batch size, and random seed need systematic sensitivity analysis.

## 16 Poster Experimental Configuration

The poster simulation uses the following concrete setting:

| Quantity                  | Value                 |
|---------------------------|-----------------------|
| trials                    | 200                   |
| trial length              | 100 bins              |
| bin size                  | $\Delta t = 0.01$ s   |
| trial duration            | 1 s                   |
| conditions                | 8 circular directions |
| latent dimension          | $k = 3$               |
| neurons per population    | $N = 80$              |
| temporal persistence      | $\phi = 0.4$          |
| latent noise scale        | $\sigma = 0.15$       |
| directional loading scale | 3.0                   |
| extra loading scale       | 0.051                 |
| nonlinearity              | softplus              |
| overdispersion            | 0.25                  |
| refractory duration       | mean 2 bins, sd 1 bin |
| burst probability         | 0.05                  |
| burst window              | 3 bins                |
| earlier population lag    | 0 bins                |
| delayed population lag    | 10 bins = 100 ms      |
| window size               | 10 bins               |
| stride                    | 1 bin                 |
| padding                   | centered zero padding |
| embedding dimension       | 3                     |
| contrastive temperature   | 0.2                   |
| soft similarity scale     | 0.5                   |
| time/label weights        | 0.5/0.5               |

For downstream analysis, each method can be exported as a trial-level structure with one element per trial. For each trial, the relevant quantities are:

- the trial direction;
- the earlier population spike matrix,  $80 \times 100$ ;
- the delayed population spike matrix,  $80 \times 100$ ;
- the trial time vector,  $1 \times 100$ ;
- the earlier recovered embedding,  $3 \times 100$ ;
- the delayed recovered embedding,  $3 \times 100$ .

The important point is conceptual rather than file-format specific: the same trial contains both the generated spike counts and the recovered manifold for the earlier and delayed populations.

## 17 Known Limitations

The current spike-count observation model is still simplified:

- overdispersion, refractoriness, and bursts are phenomenological corrections rather than mechanistic biophysical models;
- shared population noise and neuron-neuron correlations are still not explicitly modeled;
- neuron-specific spike variability is limited;
- refractory and burst parameters are currently simple distributional controls rather than fitted parameters.

## 18 Selected References

- Dayan, P. and Abbott, L. F. (2001). *Theoretical Neuroscience: Computational and Mathematical Modeling of Neural Systems*. MIT Press.
- Jolliffe, I. T. (2002). *Principal Component Analysis*. Springer.
- LeCun, Y., Bengio, Y., and Hinton, G. (2015). Deep learning. *Nature*, 521, 436–444. <https://doi.org/10.1038/nature14539>
- Vaswani, A., et al. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*. <https://arxiv.org/abs/1706.03762>
- van den Oord, A., Li, Y., and Vinyals, O. (2018). Representation learning with contrastive predictive coding. <https://arxiv.org/abs/1807.03748>
- Gallego, J. A., et al. (2023). Aligning latent representations of neural activity. *Nature Biomedical Engineering*, 7, 337–343. <https://doi.org/10.1038/s41551-022-00962-7>
- Pezzulo, G., Donnarumma, F., Ferrari-Toniolo, S., Cisek, P., and Battaglia-Mayer, A. (2022). Shared population-level dynamics in monkey premotor cortex during solo action, joint action and action observation. *Progress in Neurobiology*, 210, 102214. <https://doi.org/10.1016/j.pneurobio.2022.102214>
- Schoenemann, P. H. (1966). A generalized solution of the orthogonal Procrustes problem. *Psychometrika*, 31, 1–10. <https://doi.org/10.1007/BF02289451>
- Kriegeskorte, N., Mur, M., and Bandettini, P. (2008). Representational similarity analysis: connecting the branches of systems neuroscience. *Frontiers in Systems Neuroscience*, 2, 4. <https://doi.org/10.3389/neuro.06.004.2008>
- Haxby, J. V., et al. (2011). A common, high-dimensional model of the representational space in human ventral temporal cortex. *Neuron*, 72, 404–416. <https://doi.org/10.1016/j.neuron.2011.08.029>
- Kornblith, S., Norouzi, M., Lee, H., and Hinton, G. (2019). Similarity of neural network representations revisited. *Proceedings of ICML*. <https://arxiv.org/abs/1905.00414>